

Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Integrante	Alexander Sánchez
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente-servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guía practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación

-
- IDE de programación
 - Archivo de texto

5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos)

Este módulo se desarrolla de forma independiente

Actividad

1. Crear un archivo:
servidor.py
2. Crear un archivo auxiliar:
entrada_servidor.txt
3. Implementar un programa que:
 - o Revise periódicamente el archivo entrada_servidor.txt.
 - o Detecte si existe contenido nuevo.
 - o Procese el mensaje:
 - Ejemplo: convertir texto a mayúsculas o invertirlo.
 - o Escriba el resultado
en: salida_servidor.txt
4. Condiciones:
 - o Debe ejecutarse continuamente (simulación de servicio activo).
 - o No debe reprocesar el mismo mensaje.
 - o Debe manejar archivos vacíos.

Parte 2: Desarrollo del CLIENTE (1 hora)

Se desarrolla sin necesidad de ver el código del servidor

Actividad

1. Crear archivo:
cliente.py
2. Implementar un programa que:
 - o Solicite un mensaje al usuario.
 - o Escriba el mensaje
en: entrada_servidor.txt
 - o Espere un tiempo (simulando latencia).
 - o Lea la respuesta
desde: salida_servidor.txt
 - o Muestre el resultado en pantalla.
3. Condiciones:
 - o No debe asumir cómo funciona el servidor internamente.
 - o Solo debe conocer los archivos de entrada y salida.

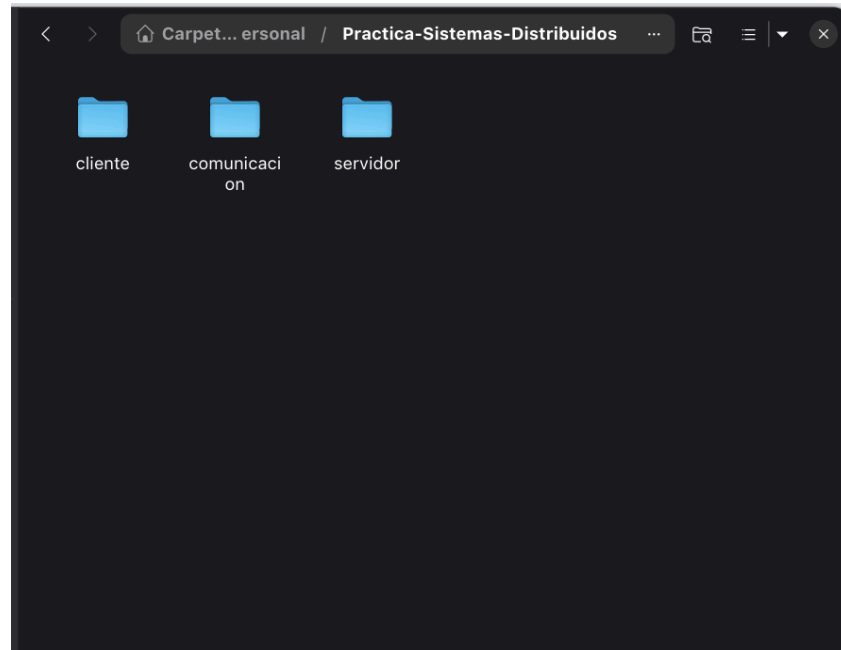


Fig.1 Carpetas creadas

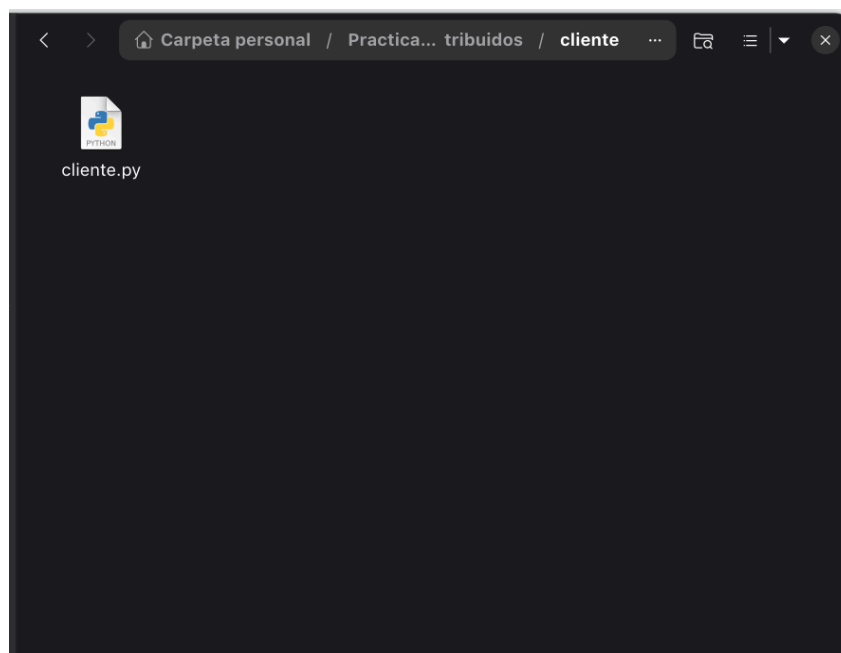
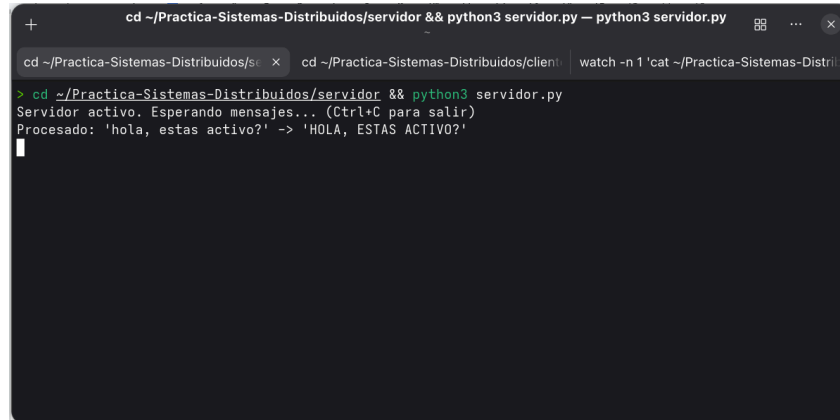


Fig.2 Archivos-Cliente-Servidor-Creados

Parte 3: Integración del sistema (25 minutos)

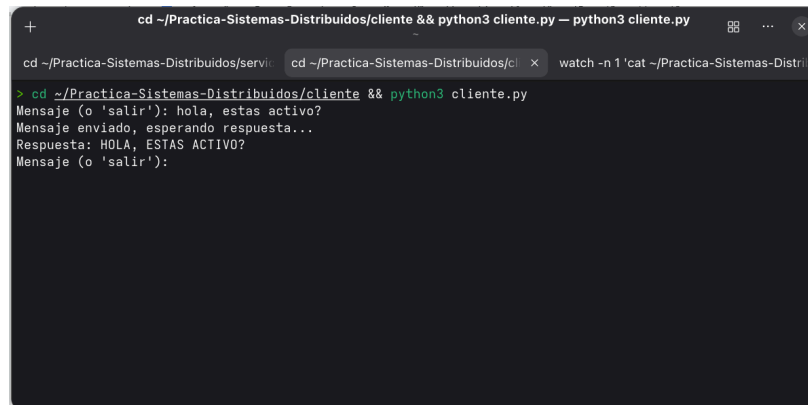
1. Ejecutar primero el servidor.



```
cd ~/Practica-Sistemas-Distribuidos/servidor && python3 servidor.py -- python3 servidor.py
cd ~/Practica-Sistemas-Distribuidos/servidor x cd ~/Practica-Sistemas-Distribuidos/cliente x watch -n 1 'cat ~/Practica-Sistemas-Distribuidos/servidor.log'
> cd ~/Practica-Sistemas-Distribuidos/servidor && python3 servidor.py
Servidor activo. Esperando mensajes... (Ctrl+C para salir)
Procesado: 'hola, estas activo?' -> 'HOLA, ESTAS ACTIVO?'
```

Fig.3 Servidor iniciado

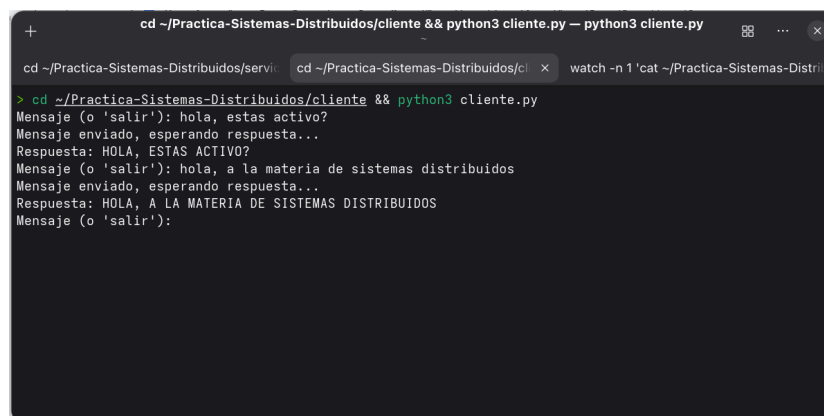
2. Ejecutar el cliente.



```
cd ~/Practica-Sistemas-Distribuidos/cliente && python3 cliente.py -- python3 cliente.py
cd ~/Practica-Sistemas-Distribuidos/servidor x cd ~/Practica-Sistemas-Distribuidos/cliente x watch -n 1 'cat ~/Practica-Sistemas-Distribuidos/servidor.log'
> cd ~/Practica-Sistemas-Distribuidos/cliente && python3 cliente.py
Mensaje (o 'salir'): hola, estas activo?
Mensaje enviado, esperando respuesta...
Respuesta: HOLA, ESTAS ACTIVO?
Mensaje (o 'salir'):
```

Fig.4 Cliente iniciado

3. Enviar varios mensajes.

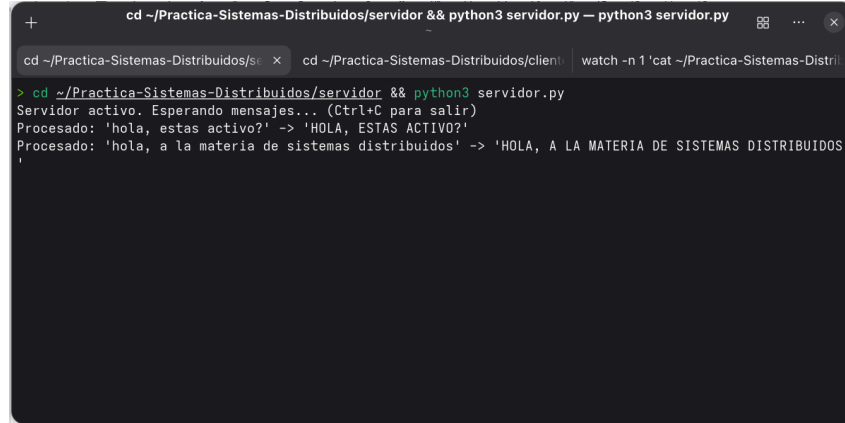


```
cd ~/Practica-Sistemas-Distribuidos/cliente && python3 cliente.py -- python3 cliente.py
cd ~/Practica-Sistemas-Distribuidos/servidor x cd ~/Practica-Sistemas-Distribuidos/cliente x watch -n 1 'cat ~/Practica-Sistemas-Distribuidos/servidor.log'
> cd ~/Practica-Sistemas-Distribuidos/cliente && python3 cliente.py
Mensaje (o 'salir'): hola, estas activo?
Mensaje enviado, esperando respuesta...
Respuesta: HOLA, ESTAS ACTIVO?
Mensaje (o 'salir'): hola, a la materia de sistemas distribuidos
Mensaje enviado, esperando respuesta...
Respuesta: HOLA, A LA MATERIA DE SISTEMAS DISTRIBUIDOS
Mensaje (o 'salir'):
```

Fig.5 Pruebas-Cliente

4. Verificar:

- o Flujo correcto
- o Respuesta adecuada



```
cd ~/Practica-Sistemas-Distribuidos/servidor && python3 servidor.py - python3 servidor.py
cd ~/Practica-Sistemas-Distribuidos/servidor && python3 servidor.py
Servidor activo. Esperando mensajes... (Ctrl+C para salir)
Procesado: 'hola, estas activo?' -> 'HOLA, ESTAS ACTIVO?'
Procesado: 'hola, a la materia de sistemas distribuidos' -> 'HOLA, A LA MATERIA DE SISTEMAS DISTRIBUIDOS'
```

Fig.6 Respuesta-Servidor

NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

6. Resultados esperados:

Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git).
- Capturas de pantalla que evidencien: (a) el archivo entrada_servidor.txt siendo llenado por el cliente, (b) salida_servidor.txt con el resultado procesado, (c) al menos 3 mensajes procesados correctamente.
- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).

7. Preguntas de Control:

¿Qué significa que cliente y servidor estén desacoplados?

Significa que cada uno funciona por su cuenta, sin necesidad de saber cómo está hecho el otro. Solo se ponen de acuerdo en cómo se van a comunicar (los nombres de los archivos y el formato de los mensajes). Así, si el servidor cambia la forma de procesar el texto, el cliente sigue funcionando igual.

¿Qué ventajas tiene desarrollar componentes por separado?

- Se pueden hacer en paralelo por distintos equipos.
- Es más fácil probar cada parte sola.
- Si se cambia un componente, el otro no se rompe mientras se respeta la interfaz.
- Se facilita el mantenimiento y se puede reutilizar.
- Los errores se localizan más rápido.

¿Qué problemas pueden surgir al integrar sistemas independientes?

- Que no coincidan los formatos de los mensajes.
- Que los nombres de los archivos no sean los mismos.
- Que el cliente espere más rápido de lo que el servidor responde.
- Que haya conflictos al escribir o leer los archivos.
- Que sea difícil encontrar dónde está el error, si en el cliente, el servidor o en la comunicación.

¿Qué sucede si el servidor no está ejecutándose?

El cliente escribe su mensaje, pero nadie lo procesa. Pasado el tiempo de espera, avisa que no hubo respuesta. El mensaje no se pierde, queda guardado y se atenderá cuando el servidor se encienda.

¿Cómo se simula la comunicación en esta práctica?

Se usan archivos de texto como si fueran el canal. El cliente escribe en uno, el servidor lo revisa cada cierto tiempo, lo procesa y responde en otro archivo. El cliente luego lee esa respuesta. El tiempo de espera simula la latencia de una red.

¿Qué limitaciones tiene este modelo frente a uno real?

- No es en tiempo real, depende de la revisión periódica.
- El servidor revisa aunque no haya nada nuevo, lo que es poco eficiente.
- Puede haber conflictos si varios escriben a la vez.
- Ambos deben estar en la misma máquina o compartir el mismo disco.
- No hay seguridad ni confirmación de recepción.
- No escala bien, los archivos se hacen cada vez más grandes.

¿Cómo se podría mejorar la sincronización?

- Usar sockets TCP o HTTP para comunicación directa.
- Usar colas de mensajes para que sea más confiable.
- Poner bloqueos en los archivos para evitar choques.
- Dar un ID a cada mensaje para saber cuál respuesta corresponde.
- Usar notificaciones en vez de revisar todo el tiempo.
- Agregar confirmaciones y reintentos.

¿Qué pasaría si múltiples clientes usan el sistema?

Se mezclarían los mensajes en el mismo archivo y las respuestas se confundirían. Un cliente podría leer la respuesta de otro. Para evitarlo se necesitarían identificadores, archivos separados o, mejor aún, usar colas de mensajes o sockets.

8. Evaluación

NO APLICA

9. Bibliografía

Semana 2. Guaman, Jose. 2026



10. Elaboración y Aprobación

Elaborado por	José Guamán Docente	Validar únicamente en FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera Técnico Docente	Validar únicamente en FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero Director de Carrera	Firmado electrónicamente por: Edison L Coronel Romero Validar únicamente con FirmaEC